Module Code & Module Title

CC5067NI Smart Data Discovery

60% Individual Coursework

Submission: Final Submission

Academic Semester: Spring Semester 2026

Credit: 15 credit semester long module

Student Name: Eijkeyal Pakhrin

London Met ID: 24**49**

College ID: NP0***********022**

Assignment Due Date: Thursday, May 11, 2026

Assignment Submission Date: Thursday, May 11, 2026

Submitted To: Alish K.C

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded

Table of Contents

Table of Figures

1. Introduction

Network intrusion detection is a critical component of today's modern cybersecurity infrastructure. In this technological era, the internet is widely spread worldwide, and with the rapid growth of the internet, cloud computing, and online communication, large volumes of network traffic are generated every second. As a result, it has been difficult to manage and monitor manually detected malicious activities such as unauthorized access, data breaches, and infiltration attacks. Therefore, analyzing network traffic data with the help of programming and data analysis techniques has become an important way to identify and monitor abnormal behavior in computer networks.

This coursework focuses on analyzing network traffic data in order to distinguish between benign (normal) and malicious (attack) traffic patterns using Python-based data analysis techniques and tools such as NumPy, pandas, matplotlib, seaborn, etc. The dataset used in this coursework contains several network flow features such as destination port, flow duration, packet counts, packet lengths, and other statistical metrics that describe data flow between devices in a network. By understanding these features, it becomes easier to identify patterns that differentiate normal traffic from attack traffic.

The main motto of this coursework is to apply programming skills and implement problem-solving skills to solve with real-world data and demonstrate analytical thinking and data interpretation abilities. These analytical skills help in preparing the dataset for future machine learning models that can automatically detect network intrusions more efficiently.

1.1. Aim

The aim of this coursework is to analyze network traffic data using Python libraries and Jupiter Notebook to understand the network flow characteristics, identify differences between normal and attack traffic, and prepare the datasets for further machine learning classification.

1.2. Objectives

- To understand the structure of the network traffic dataset.
- To load the dataset into a pandas Data Frame and other libraries for the analysis.
- To clean the dataset and handle the empty values and duplicate records.
- To prepare or clean the dataset for the further machine learning models.
- To visualize the dataset using charts such as bar charts, pie charts and boxplots.
- To compare and identify normal traffic and attack traffic using statistical and visual analysis.

2. Background & context

This coursework focuses on the increasing amount of unusual network traffic on the internet, which has led to a rise in cyberattacks. To protect networks and sensitive information, organizations use intrusion Detection Systems (IDS). As an analysis, our task is to study how network traffic moves through a system. The given dataset contains network flow characteristics such as packet counts, packet sizes, flow duration and data transmission rates. By analyzing this data, it becomes easier to identify unusual patterns that may indicate cyberattacks, such as infiltration attempts and other malicious activities.

2.1. Problem Statement

The main challenge of this coursework is to understand and organize the network traffic data in a proper structured way so that it can be used for the further data mining and machine learning classifications. This includes several steps such as cleaning dataset, pre-processing the data and identifying the important features and performing statistical analysis to compare normal traffic and attack traffic.

2.2. Dataset Information

This coursework uses a network traffic dataset that contains different network flow features such as destination port, flow duration, packet counts, packet lengths, and other statistical values. Each row in the dataset represents one network flow and includes a label that shows whether the traffic is BENIGN (normal traffic) or infiltration (attack traffic). The dataset also contains a large number of records, which makes it useful for data analysis and future machine learning applications.

3. Tools and Libraries Used

In this coursework I used Python and several data analysis libraries. Those libraries were used to load, clean, analyze, and visualize the network traffic dataset.

3.1. Python

Python is an interpreted, object oriented, high programming language with dynamic semantics. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports external library like NumPy, Pandas, Matplotlib, Seaborn, SciPy through a sophisticated ecosystem that manages installation, dependency resolution and efficient code execution (Python, n.d.).

Figure 1 Official Logo of the Python Programming

3.2. Jupiter Notebook

A Jupiter Notebook is an interactive web application used to create and share documents that contain live code, equations, visualizations and narrative text. Jupiter Notebook is flexible interface it allows users to configure and arrange the workflows in data science, scientific computing, computational journalism and machine learning (Jupyter, n.d.).

Figure 2 Official Logo of the Jupyter Notebook Environment

3.3. NumPy

NumPy is a library known as Numerical Python used for numerical operations and mathematical calculations such as mean, standard deviation, skewness, and kurtosis. (GeeksforGeek, 2026).

Figure 3 Official Logo of the NumPy Scientific Computing Library

3.4. Pandas

Pandas is a python library that offers fast and flexible data structures for handling structured or labeled data. Basically, pandas is used for data preparation such as removing missing values, finding duplicates and cleaning dataset. It makes data analysis and manipulation simple and efficient data (Pandas, 2026).

Figure 4 Official Logo of the Pandas Data Analysis Library

3.5. Matplotlib

Matplotlib is python library used to visualize data and help draw meaningful statistical insights. It is widely used to create visualizations, such as bar charts, pie charts and box plots, to make it easier to understand the data (Scalar, 2022).

Figure 5 Official Logo of the Matplotlib Visualization Library

3.6. Seaborn

Seaborn is also a Python library used to create advanced and visually clear statistical plots such as boxplots and correlation heatmaps, which helps to identify relationships between different features (Seaborn, 2025).

Figure 6 Official Logo of the Seaborn Data Visualization Library

3.7. SciPy

Scipy is a collection of mathematical algorithms and convenience functions built on NumPy. It adds significant power to python by providing the user with high-commands and classes for manipulating and visualizing data. It is used for high level scientific and technical computing in python (SciPy, n.d.).

Figure 7 Official Logo of the SciPy Scientific Computing Library

3.8. Scikit Learn

Scikit Learn is python library used for machine learning and data analysis. It's built on top of fundamental scientific libraries like Numpy, SciPy and Matplotlib. It provides a standardized and users friendly interface for the implementing traditional machine learning algorithm and data science (Scikit Learn, n.d.).

Figure 8 Official Logo of the Scikit-learn Machine Learning Library

4. Data Understanding

The dataset used in this coursework contains network traffic flow data is that is used to analyze normal and attack traffic. The dataset includes 488,115 rows and 80 columns which represent a large amount network traffic data.

The dataset contains several important features such as Destination Port, Flow Duration, Total Forward Packets, Total Backward Packets, Total Length of Forward Packets and Total Length of Backward Packets. The Destination Port shows where the traffic is being sent, Flow Duration shows how long the connection lasts, Total Forward and Backward Packets show how many packets are sent and received, and the packet length features show how much data is transferred. These features help to understand how data moves through the network and how different types of traffic behave.

[54]:

	Unnamed: 0	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	...	min_seg_size_forward	Active Mean	Active Std	Active Max	Active Min	Idle Mean
0	0	53	62171	2	2	78	164	39	39	39.00	...	32	0.00000	0.00000	0	0	0.0
1	1	2710	48	2	0	4	0	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0
2	2	443	4792909	5	1	135	46	46	6	27.00	...	20	0.00000	0.00000	0	0	0.0
3	3	80	115596470	75	82	342	118155	342	0	4.56	...	32	22092.63636	22835.59127	90944	14999	10000000.0
4	4	2910	14	2	2	4	12	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
488110	488110	53	238	2	2	86	228	43	43	43.00	...	32	0.00000	0.00000	0	0	0.0
488111	488111	8899	3	2	0	4	0	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0
488112	488112	53	250	2	2	82	206	41	41	41.00	...	20	0.00000	0.00000	0	0	0.0
488113	488113	53	70581019	2	2	117	231	66	51	58.50	...	32	41306.00000	0.00000	41306	41306	70300000.0
488114	488114	5679	121	2	2	4	12	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0

488115 rows × 80 columns

Figure 9 Data Understanding

4.1. Explanation of Important Columns

Destination Port: This destination port columns represents the port number where the network traffic is directed. Different services and applications use different port numbers for communications.

Flow Duration: Flow Durations columns shows the total duration of a network flow from the start of the connections to the end. Larger values indicate the longer communications sessions.

Total Fwd Packets: This column indicates the total number of packets transmitted from the source device to the destination device during the communication process.

Total Backward Packets: It represents the total number of packets sent from the destination back to the source. It helps to understand the two-way communication between devices.

Total Length of Fwd Packets: It shows the total amount of data, measured in bytes, transmitted in the forward direction. Higher values indicate larger data transfer from the source to destinations.

Total Length of Bwd Packets: It indicates the total size of data transferred in the backward direction from the destination to the source.

Fwd Packet Length Mean: This column represents the average size of forward packets sent during the flow. It helps to analyze packet transmissions behavior in the forward direction.

Bwd Packet Length Mean: It Indicates the average size of packets transferred in the backward direction and helps to compare traffic behavior between both directions.

Flow Bytes: This column Shows the rate of data transfer in bytes per second during the network flow. It indicates how quickly data is transmitted within the connection.

Packet Length Mean: It represents the average size of all packets in the flow. This helps in understanding the general packet size distributions in the dataset.

Packet Length std: This column measures the variation or spread of packet in the flow. A larger value indicates higher differences between packet sizes.

Average Packet Size: It Show the average size of packets transmitted during the connection. It provides a summary of packet transmission characteristics.

Active Mean: It represents the average time the connection remains active and continuously transmits data.

Idle Mean: Represents the average time the connection remains idle (no data transmission).

Label: This column indicates the target category of the dataset whether the traffic is BENIGN (Normal Traffic) or an Attack Traffic.

4.2. Dataset Structure Using info

```
df.info()
[38] 0.0s                                                    Python

<class 'pandas.DataFrame'>
RangeIndex: 488115 entries, 0 to 488114
Data columns (total 80 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Unnamed: 0                   488115 non-null  int64
 1   Destination Port             488115 non-null  int64
 2   Flow Duration                488115 non-null  int64
 3   Total Fwd Packets            488115 non-null  int64
 4   Total Backward Packets       488115 non-null  int64
 5   Total Length of Fwd Packets  488115 non-null  int64
 6   Total Length of Bwd Packets  488115 non-null  int64
 7   Fwd Packet Length Max        488115 non-null  int64
 8   Fwd Packet Length Min        488115 non-null  int64
 9   Fwd Packet Length Mean       488115 non-null  float64
 10  Fwd Packet Length Std        488115 non-null  float64
 11  Bwd Packet Length Max        488115 non-null  int64
 12  Bwd Packet Length Min        488115 non-null  int64
 13  Bwd Packet Length Mean       488115 non-null  float64
 14  Bwd Packet Length Std        488115 non-null  float64
 15  Flow Bytes/s                 488079 non-null  float64
 16  Flow Packets/s               488115 non-null  float64
 17  Flow IAT Mean                488115 non-null  float64
 18  Flow IAT Std                 488115 non-null  float64
 19  Flow IAT Max                 488115 non-null  int64
...
 78  Idle Min                     488115 non-null  int64
 79  Label                        488115 non-null  str
dtypes: float64(24), int64(55), str(1)
memory usage: 297.9 MB
```

Figure 10 Descriptions of the Dataset

Function Explanation

The **df.info()** functions is used to display the overall structure and technical summary of the dataset It shows the details such as:

- Total number of rows and columns
- Column names
- Number of non-null values
- Data types of each column

This function helps to understand the dataset before performing data cleaning before analyzing insights.

Output Explanation

The given output shows that the dataset contains 4,88,115 and 80mcolumns, which indicates a very large network traffic dataset suitable for data analysis and further machine learning tasks. Most columns are numerical data types such as int and float while the label column is categorical (str).

5. Data Preparation

5.1. Load dataset

Pandas is first imported to work with the CSV dataset and then loaded into a Data Frame using **df=pd.read_csv('network_dataset.csv').**

Function Explanation

The df command is used to display the dataset after loading it into the pandas DataFrame. It helps to preview the structure and contents of the datastet.

Output Explanation

The given output shows the network traffic dataset containing multiple rows and 80 columns with different network flow features and traffic label.

```
[79]: import pandas as pd

[84]: #Load the Dataset
      df = pd.read_csv('network_traffic.csv')

[83]: df
```

[83]:

	Unnamed: 0	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	...	min_seg_size_forward	Active Mean	Active Std	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	Label
0	0	53	62171	2	2	78	164	39	39	39.00	...	32	0.00000	0.00000	0	0	0.0	0.000000	0	0	BENIGN
1	1	2710	48	2	0	4	0	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0	0.000000	0	0	Infiltration
2	2	443	4792909	5	1	135	46	46	6	27.00	...	20	0.00000	0.00000	0	0	0.0	0.000000	0	0	BENIGN
3	3	80	115596470	75	82	342	118155	342	0	4.56	...	32	22092.63636	22835.59127	90944	14999	10000000.0	2701.650839	10000000	9997164	BENIGN
4	4	2910	14	2	2	4	12	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0	0.000000	0	0	BENIGN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
488110	488110	53	238	2	2	86	228	43	43	43.00	...	32	0.00000	0.00000	0	0	0.0	0.000000	0	0	BENIGN
488111	488111	8899	3	2	0	4	0	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0	0.000000	0	0	Infiltration
488112	488112	53	250	2	2	82	206	41	41	41.00	...	20	0.00000	0.00000	0	0	0.0	0.000000	0	0	BENIGN
488113	488113	53	70581019	2	2	117	231	66	51	58.50	...	32	41306.00000	0.00000	41306	41306	70300000.0	0.000000	70300000	70300000	BENIGN
488114	488114	5679	121	2	2	4	12	2	2	2.00	...	24	0.00000	0.00000	0	0	0.0	0.000000	0	0	BENIGN

488115 rows × 80 columns

Figure 11 Loading Dataset

5.2. Display last 50 records

After loading the data in data frame, I used **df.tail(50)** command to display the last 50 records in the dataset.

```
#Display last 50 Records
df.tail(50)
```

	Unnamed: 0	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	...	min_seg_size_forward	Active Mean	Active Std	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	Label
488065	488065	53	70591	2	2	66	194	33	33	33.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488066	488066	443	5572560	8	6	386	5205	205	0	48.250000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488067	488067	3013	49	2	0	4	0	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488068	488068	53	122130	2	2	80	272	40	40	40.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488069	488069	53	98106274	2	2	114	211	58	56	57.000000	...	20	30943.00000	0.00000	30943	30943	9.800000e+07	0.00000	98000000	98000000	Infiltration
488070	488070	53	506	1	1	53	113	53	53	53.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488071	488071	53	3957631	2	2	120	260	79	41	60.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488072	488072	9102	2	2	0	4	0	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488073	488073	2135	111	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488074	488074	1048	49	2	0	4	0	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488075	488075	54240	55	1	1	6	6	6	6	6.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488076	488076	1049	73	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488077	488077	443	4	2	0	12	0	6	6	6.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488078	488078	53	172	2	2	80	112	40	40	40.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488079	488079	443	79063	1	2	0	0	0	0	0.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488080	488080	51425	4	2	0	0	0	0	0	0.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488081	488081	5120	85	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488082	488082	39930	4	2	0	0	0	0	0	0.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488083	488083	443	5871633	7	4	635	168	517	0	90.714286	...	32	150944.00000	0.00000	150944	150944	5.720685e+06	0.00000	5720685	5720685	BENIGN
488084	488084	10616	99	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488085	488085	443	115338719	22	18	1317	7539	864	0	59.863636	...	20	84994.45455	93143.49644	365777	56220	9.992959e+06	27637.53213	10000000	9910557	BENIGN
488086	488086	60928	52	1	1	0	0	0	0	0.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488087	488087	22	1340965	41	42	2728	6634	456	0	66.536585	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488088	488088	443	1142446	78	107	997	227837	517	0	12.782051	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488089	488089	53	67977	2	2	70	130	35	35	35.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488090	488090	53	32534951	2	2	130	297	79	51	65.000000	...	32	475.00000	0.00000	475	475	3.250000e+07	0.00000	32500000	32500000	Infiltration
488091	488091	53	31361	2	2	70	198	35	35	35.000000	...	40	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488092	488092	53	228451	2	2	62	240	31	31	31.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488093	488093	53	216	2	2	78	160	39	39	39.000000	...	40	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488094	488094	6302	47740	2	0	6	0	6	0	3.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488095	488095	53	210	2	2	70	130	35	35	35.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488096	488096	443	5322619	5	5	3471	1132	1993	6	694.200000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488097	488097	53	118556	2	2	76	300	38	38	38.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488098	488098	389	69	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488099	488099	5405	72	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN

Figure 12 Displaying Last 50 Records

Function Explanation

The **df.tail(50)** function is used to display the last 50 dataset. It helps to examine the ending records and verify that the dataset has been loaded correctly and successfully.

Output Explanation

The given output shows the last 50 records of the network traffic dataset along with different flow features and traffic labels such as BENIGN and Infiltration.

5.3. Creating a new Data Frame with following columns

A list of important columns is created and stored in the variable columns. Then create a new data frame called **new_df** is created by selecting only those columns form the original dataset. This step extracts unnecessary columns and focus only on the important extracted feature that for further analysis.

```
[87]: #Create a new dataframe
columns = ['Destination Port', 'Flow Duration', 'Total Fwd Packets','Total Backward Packets',  'Total Length of Fwd Packets','Total Length of Bwd Packets','Fwd Packet Length Mean','Bwd Packet Length Mean',
       'Flow Bytes/s', 'Flow Packets/s','Packet Length Mean','Packet Length Std', 'Average Packet Size','Active Mean', 'Idle Mean', 'Label']
new_df = df[columns]

[88]: new_df
```

[88]:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean	Label
0	53	62171	2	2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000	0.00000	0.0	BENIGN
1	2710	48	2	0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000	0.00000	0.0	Infiltration
2	443	4792909	5	1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333	0.00000	0.0	BENIGN
3	80	115596470	75	82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962	22092.63636	10000000.0	BENIGN
4	2910	14	2	2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000	0.00000	0.0	BENIGN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
488110	53	238	2	2	86	228	43.00	114.000000	1.319328e+06	16806.722690	71.400000	38.888302	89.250000	0.00000	0.0	BENIGN
488111	8899	3	2	0	4	0	2.00	0.000000	1.333333e+06	666666.666700	2.000000	0.000000	3.000000	0.00000	0.0	Infiltration
488112	53	250	2	2	82	206	41.00	103.000000	1.152000e+06	16000.000000	65.800000	33.958799	82.250000	0.00000	0.0	BENIGN
488113	53	70581019	2	2	117	231	58.50	115.500000	4.930504e+00	0.056672	82.800000	30.817203	103.500000	41306.00000	70300000.0	BENIGN
488114	5679	121	2	2	4	12	2.00	6.000000	1.322314e+05	33057.851240	3.600000	2.190890	4.500000	0.00000	0.0	BENIGN

488115 rows × 16 columns

Figure 13 Creating New Data Frame

Before:

[56]:

	Unnamed: 0	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	...	min_seg_size_forward	Active Mean	Active Std	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	Label
488065	488065	53	70591	2	2	66	194	33	33	33.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488066	488066	443	5572560	8	6	386	5205	205	0	48.250000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488067	488067	3013	49	2	0	4	0	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488068	488068	53	122130	2	2	80	272	40	40	40.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488069	488069	53	98106274	2	2	114	211	58	56	57.000000	...	20	30943.00000	0.00000	30943	30943	9.800000e+07	0.00000	98000000	98000000	Infiltration
488070	488070	53	506	1	1	53	113	53	53	53.000000	...	32	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488071	488071	53	3957631	2	2	120	260	79	41	60.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488072	488072	9102	2	2	0	4	0	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488073	488073	2135	111	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488074	488074	1048	49	2	0	4	0	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488075	488075	54240	55	1	1	6	6	6	6	6.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
488076	488076	1049	73	2	2	4	12	2	2	2.000000	...	24	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
488077	488077	443	4	2	0	12	0	6	6	6.000000	...	20	0.00000	0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration

Figure 14 Old Dataset

After:

```
[57]: #Create a new dataframe
columns = ['Destination Port', 'Flow Duration',
        'Total Fwd Packets','Total Backward Packets',
        'Total Length of Fwd Packets','Total Length of Bwd Packets',
        'Fwd Packet Length Mean','Bwd Packet Length Mean',
        'Flow Bytes/s', 'Flow Packets/s','Packet Length Mean',
        'Packet Length Std', 'Average Packet Size','Active Mean', 'Idle Mean', 'Label']

new_df = df[columns]

[58]: new_df
```

[58]:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean	Label
0	53	62171	2	2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000	0.00000	0.0	BENIGN
1	2710	48	2	0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000	0.00000	0.0	Infiltration
2	443	4792909	5	1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333	0.00000	0.0	BENIGN
3	80	115596470	75	82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962	22092.63636	10000000.0	BENIGN
4	2910	14	2	2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000	0.00000	0.0	BENIGN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
488110	53	238	2	2	86	228	43.00	114.000000	1.319328e+06	16806.722690	71.400000	38.888302	89.250000	0.00000	0.0	BENIGN
488111	8899	3	2	0	4	0	2.00	0.000000	1.333333e+06	666666.666700	2.000000	0.000000	3.000000	0.00000	0.0	Infiltration
488112	53	250	2	2	82	206	41.00	103.000000	1.152000e+06	16000.000000	65.800000	33.958799	82.250000	0.00000	0.0	BENIGN
488113	53	70581019	2	2	117	231	58.50	115.500000	4.930504e+00	0.056672	82.800000	30.817203	103.500000	41306.00000	70300000.0	BENIGN
488114	5679	121	2	2	4	12	2.00	6.000000	1.322314e+05	33057.851240	3.600000	2.190890	4.500000	0.00000	0.0	BENIGN

488115 rows × 16 columns

Figure 15 New Dataset

Function Explanation

A list of selected important columns is stored in the variable columns and then create a new DataFrame called **new_df** is created and put the selected columns from the original dataset using **df[columns].**

Output Explanation

The given output shows a new dataset containing only the selected network traffic features required for further analysis. This helps them to simplify the dataset and focus on selected important Columns

5.4. Checking missing values

The **new_df.isnull().sum()** command is used to check missing(NaN) values.

```
[89]: #Checking Missing Values
      new_df.isnull().sum()
```

```
[89]: Destination Port              0
      Flow Duration                 0
      Total Fwd Packets             0
      Total Backward Packets        0
      Total Length of Fwd Packets   0
      Total Length of Bwd Packets   0
      Fwd Packet Length Mean        0
      Bwd Packet Length Mean        0
      Flow Bytes/s                 36
      Flow Packets/s                0
      Packet Length Mean            0
      Packet Length Std             0
      Average Packet Size           0
      Active Mean                   0
      Idle Mean                     0
      Label                         0
      dtype: int64
```

Figure 16 Checking Missing values

Function Explanation

The function **isnull()** is used to check for missing values int the dataset, while the **sum()** functions is used to counts the total number of missing values in each column.

Output Explanation

The given output shows that the most columns do not contain missing values. However, the Flow Byte/s column contains 36 missing values, which need to remove during the data cleaning before analyzing the insights.

5.5. Removing missing values

After finding missing contains **new_df.dropna()** command is used to remove all rows that contain missing values. This ensures that the dataset is clean and ready for the further analysis.

```
[113]: #Remove Missing values
       new_df = new_df.dropna()
       new_df.isnull().sum()

[113]: Destination Port               0
       Flow Duration                  0
       Total Fwd Packets              0
       Total Backward Packets         0
       Total Length of Fwd Packets    0
       Total Length of Bwd Packets    0
       Fwd Packet Length Mean         0
       Bwd Packet Length Mean         0
       Flow Bytes/s                   0
       Flow Packets/s                 0
       Packet Length Mean             0
       Packet Length Std              0
       Average Packet Size            0
       Active Mean                    0
       Idle Mean                      0
       Label                          0
       dtype: int64
```

Figure 17 Removing Missing values

Before:

```
[89]: #Checking Missing Values
      new_df.isnull().sum()

[89]: Destination Port               0
      Flow Duration                  0
      Total Fwd Packets              0
      Total Backward Packets         0
      Total Length of Fwd Packets    0
      Total Length of Bwd Packets    0
      Fwd Packet Length Mean         0
      Bwd Packet Length Mean         0
      Flow Bytes/s                  36
      Flow Packets/s                 0
      Packet Length Mean             0
      Packet Length Std              0
      Average Packet Size            0
      Active Mean                    0
      Idle Mean                      0
      Label                          0
      dtype: int64
```

Figure 18 Before Removing Null values

After:

```
[113]: #Remove Missing values
       new_df = new_df.dropna()
       new_df.isnull().sum()
```

```
[113]: Destination Port               0
       Flow Duration                  0
       Total Fwd Packets              0
       Total Backward Packets         0
       Total Length of Fwd Packets    0
       Total Length of Bwd Packets    0
       Fwd Packet Length Mean         0
       Bwd Packet Length Mean         0
       Flow Bytes/s                   0
       Flow Packets/s                 0
       Packet Length Mean             0
       Packet Length Std              0
       Average Packet Size            0
       Active Mean                    0
       Idle Mean                      0
       Label                          0
       dtype: int64
```

Figure 19 After Removing values

Function Explanation

The **df.dropna()** function is used to remove rows that contain missing values from the dataset. After removing missing values from dataset **isnull().sum()** is used again to verify that is there any null values still exist.

Output Explanation

The given output shows that all columns now have 0 missing values, that confirms now dataset has been cleaned successfully and ready for further analysis.

5.6. Checking duplicate records

The command **new_df.duplicated().sum()** displays the total number of duplicate records.

```
[114]: #Checking Duplicate Rows
       duplicate_count = new_df.duplicated().sum()
       duplicate_count

[114]: 139389
```

Figure 20 Duplicate Values

Function Explanation

The **duplicated()** function is used to identify duplicate rows in the dataset, while the **sum()** function is used to counts the total number of duplicated records.

Output Explanation

The given output shows that the dataset contains 139389 duplicate rows. Identifying duplicate records is important because it can affect the accuracy of data analysis and further machine learning models.

5.7. Displaying the number of columns and column names

The command **new_df.shape** is used to display the total number of columns in the dataset. The new **new_df.columns.tolist()** is used to display the names of columns. This ensures that only required columns are selected.

```
[108]: #Show the total number of columns
       new_df.shape

[108]: (488079, 16)

[110]: #display all column names
       new_df.columns.tolist()

[110]: ['Destination Port',
        'Flow Duration',
        'Total Fwd Packets',
        'Total Backward Packets',
        'Total Length of Fwd Packets',
        'Total Length of Bwd Packets',
        'Fwd Packet Length Mean',
        'Bwd Packet Length Mean',
        'Flow Bytes/s',
        'Flow Packets/s',
        'Packet Length Mean',
        'Packet Length Std',
        'Average Packet Size',
        'Active Mean',
        'Idle Mean',
        'Label']
```

Figure 21 Displaying Number of Columns and Column Names

Function Explanation

The **shape()** function is used to display the dimensions of the dataset in the form of rows and columns. The **columns.tolist()** function is used to display all the columns names as a list from the dataset.

Output Explanation

The given output shows that the new dataset contains 4,88,079 rows and 16 columns. It also displays the names of all selected network traffic features under for further analysis.

5.8. Renaming the Label column

The **replace ()** function is used to rename the values in the label column. The value is BENIGN is replaced with the Normal Traffic and infiltration is replaced with attack.

```
[122]: #Rename values in the label column
       new_df['Label'] = new_df['Label'].replace({'BENIGN': 'Normal Traffic','Infiltration': 'Attack'})

[123]: new_df
```

[123]:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean	Label
0	53	62171	2	2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000	0.00000	0.0	Normal Traffic
1	2710	48	2	0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000	0.00000	0.0	Attack
2	443	4792909	5	1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333	0.00000	0.0	Normal Traffic
3	80	115596470	75	82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962	22092.63636	10000000.0	Normal Traffic
4	2910	14	2	2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000	0.00000	0.0	Normal Traffic
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
488110	53	238	2	2	86	228	43.00	114.000000	1.319328e+06	16806.722690	71.400000	38.888302	89.250000	0.00000	0.0	Normal Traffic
488111	8899	3	2	0	4	0	2.00	0.000000	1.333333e+06	666666.666700	2.000000	0.000000	3.000000	0.00000	0.0	Attack
488112	53	250	2	2	82	206	41.00	103.000000	1.152000e+06	16000.000000	65.800000	33.958799	82.250000	0.00000	0.0	Normal Traffic
488113	53	70581019	2	2	117	231	58.50	115.500000	4.930504e+00	0.056672	82.800000	30.817203	103.500000	41306.00000	70300000.0	Normal Traffic
488114	5679	121	2	2	4	12	2.00	6.000000	1.322314e+05	33057.851240	3.600000	2.190890	4.500000	0.00000	0.0	Normal Traffic

488079 rows × 16 columns

Figure 22 Renaming the Label values

Before:

[58]:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean	Label
0	53	62171	2	2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000	0.00000	0.0	BENIGN
1	2710	48	2	0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000	0.00000	0.0	Infiltration
2	443	4792909	5	1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333	0.00000	0.0	BENIGN
3	80	115596470	75	82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962	22092.63636	10000000.0	BENIGN
4	2910	14	2	2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000	0.00000	0.0	BENIGN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
488110	53	238	2	2	86	228	43.00	114.000000	1.319328e+06	16806.722690	71.400000	38.888302	89.250000	0.00000	0.0	BENIGN
488111	8899	3	2	0	4	0	2.00	0.000000	1.333333e+06	666666.666700	2.000000	0.000000	3.000000	0.00000	0.0	Infiltration
488112	53	250	2	2	82	206	41.00	103.000000	1.152000e+06	16000.000000	65.800000	33.958799	82.250000	0.00000	0.0	BENIGN
488113	53	70581019	2	2	117	231	58.50	115.500000	4.930504e+00	0.056672	82.800000	30.817203	103.500000	41306.00000	70300000.0	BENIGN
488114	5679	121	2	2	4	12	2.00	6.000000	1.322314e+05	33057.851240	3.600000	2.190890	4.500000	0.00000	0.0	BENIGN

488115 rows × 16 columns

Figure 23 Before Renaming

After:

Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean	Label
2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000	0.00000	0.0	Normal Traffic
0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000	0.00000	0.0	Attack
1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333	0.00000	0.0	Normal Traffic
82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962	22092.63636	10000000.0	Normal Traffic
2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000	0.00000	0.0	Normal Traffic
...	...	...	...	...	...	...	...	...	...	...	...	...
2	86	228	43.00	114.000000	1.319328e+06	16806.722690	71.400000	38.888302	89.250000	0.00000	0.0	Normal Traffic
0	4	0	2.00	0.000000	1.333333e+06	666666.666700	2.000000	0.000000	3.000000	0.00000	0.0	Attack
2	82	206	41.00	103.000000	1.152000e+06	16000.000000	65.800000	33.958799	82.250000	0.00000	0.0	Normal Traffic
2	117	231	58.50	115.500000	4.930504e+00	0.056672	82.800000	30.817203	103.500000	41306.00000	70300000.0	Normal Traffic
2	4	12	2.00	6.000000	1.322314e+05	33057.851240	3.600000	2.190890	4.500000	0.00000	0.0	Normal Traffic

Figure 24 After Renaming

Function Explanation

The **replace()** function is used to rename values in the label column. In this dataset, BEGINI is replaced with Normal Traffic and Infiltration is replaced with Attack to make the traffic categories easier to understand.

Output Explanation

The given output shows that the label values have been successfully renamed from technical terms into more understandable categories such as Normal Traffic and Attack Traffic.

6. Data Analysis

6.1. Summary Statistics of Selected Columns

For data analysis I selected two numeric columns Flow Duration and Packet Length Mean from the dataset. Then I calculated different statistical calculations such as sum, mean, standard deviation, skewness and kurtosis from chooses numeric variable. The main purpose of purpose of doing this is to understand the overall distribution, variability and behavior of the data.

QN.3 Data Analysis

```
#Select two numeric columns
cols = ['Flow Duration','Packet Length Mean']
cols
```

[20] ✓ 0.0s

```
... ['Flow Duration', 'Packet Length Mean']
```

Figure 25 Selection of Numeric Variables

Function Explanation

A list named cols is created to store two selected numeric columns: Flow Duration and Packet Length Mean. These columns are chosen for statistical analysis.

Output Explanation

The output displays that the selected column names that will be used further calculations such as sum, Mean, Standard Deviation, Skewness, and Kurtosis.

6.2. Sum Analysis

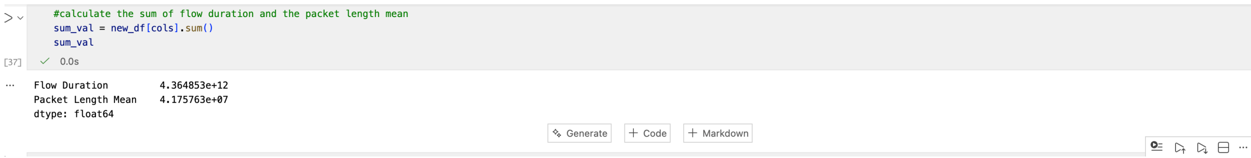


Figure 26 Sum of Selected Features

The total flow duration is extremely large compared to the packet length mean, indicating a much higher cumulative value across all records.

Function Explanation

The **sum()** function is used to calculate the total sum of values for the flow duration and packet length mean columns in the dataset.

Output Explanation

The given output show that the total values of Flow Duration and Packet Length Mean across all records. To compare two different columns, the total flow of duration is much larger than to the packet length mean, indicating a very high cumulative network connection duration in the dataset.

6.3. Mean Analysis

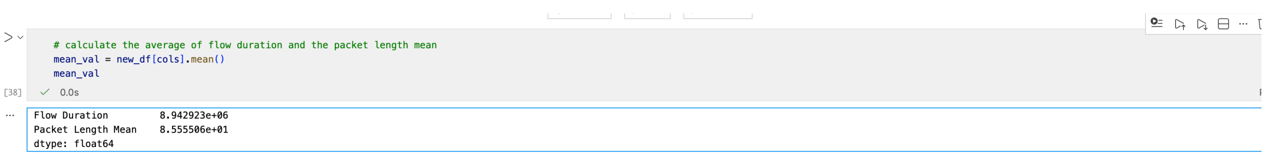


Figure 27 Mean of Selected Features

The average flow duration is significantly higher, while the packet length mean remains relatively small, showing more consistent typical values.

Function Explanation

The **mean()** function is used to calculate the average value of flow duration and packet length mean from the dataset.

Output Explanation

The output shows the average values of Flow Duration and Packet Length Mean. The given output of average flow duration is significantly giher that the packet length mean, indicating larger overall connection durations in the network traffic data.

6.4. Standard Deviation Analysis

```
#Calculate the standard deviation of flow duration and the packet length mean
std_val = new_df[cols].std()
std_val
```

```
[40] ✓ 0.0s
```

```
Flow Duration         2.748182e+07
Packet Length Mean    1.738725e+02
dtype: float64
```

Figure 28 Standard Deviation of Selected Features

The flow duration has a very high spread, indicating large variability, whereas packet length mean shows moderate variation.

Function Explanation

The **std()** function is used to calculate the standard deviation of the standard numeric columns. Standard deviation measures how much the data values vary from the average value.

Output Explanation

The given output shows that Flow Duration has a much higher standard deviation compared to Packet Length Mean, indicating that flow duration values are more widely spread and contain greater variability in the dataset.

6.5. Skewness Analysis

```
#Calculate the skewness of flow duration and the packet length mean
skew_val = new_df[cols].skew()
skew_val
```
[41] ✓ 0.0s

```
Flow Duration        3.250874
Packet Length Mean   4.935766
dtype: float64
```

Figure 29 Skewness of Selected Features

Both flow duration and packet length mean are positively skewed, indicating that most values are smaller with a few large outliers.

Function Explanation

The **skew()** function is used to measure the skewness of the two selected columns. This skewness indicates the data distribution is symmetric or biased toward one side.

Output Explanation

The given output shows that both Flow Duration and Packet Length Mean have positive skewness values, indicating that most data values are similar with a few extremely large values present in the dataset.

6.6. Kurtosis Analysis

```
#Calculate the kurtosis of flow duration and the packet length mean
kurt_val = new_df[cols].kurt()
kurt_val
```
[42] ✓ 0.0s

```
Flow Duration         9.246791
Packet Length Mean   34.610512
dtype: float64
```

Both variables show high kurtosis, especially packet length mean, indicating heavy tails and the presence of extreme values.

Function Analysis

The **kurt()** functions is used to calculate the kurtosis of the Flow Duration and Packet Length Mean columns. Kurtosis measures the presence of extreme values or outliers in the distributions.

Output Explanation

The given output shows that high kurtosis values for both Flow Duration and Packet Length Mean, indicating that the dataset contains extreme values and heavy tails, especially in Packet Length Mean.

Figure 30 Kurtosis of Selected Features

6.7. Correlation Analysis of Numeric Variable

The results show that some features are highly correlated especially in Packet Length Mean and Average Packet Size. This means they provide similar information and other strong correlations indicates that relationships between packet counts and packet sizes.

```python
#Correlation of numeric columns
corr_matrix = new_df.corr(numeric_only = True)
```

[25] Python

```python
#Display top 5 correlated Features
correlated = new_df.corr(numeric_only = True).unstack().sort_values(ascending = False)
correlated = correlated[correlated < 1]
correlated.drop_duplicates().head(5)
```

[26] Python

```
Packet Length Mean          Average Packet Size          0.995068
Total Length of Bwd Packets Total Backward Packets       0.961480
Total Fwd Packets           Total Backward Packets       0.949848
                            Total Length of Bwd Packets  0.920007
Packet Length Mean          Packet Length Std            0.887792
dtype: float64
```

Generate + Code + Markdown

Figure 31 Top 5 Correlated Features

Function Explanation

The corr() function is used to calculate the correlation between numeric columns in the dataset. The sort_values() function arranges the correlation values in descending order to identify the strongest relationships between features.

Output Explanation

The given output shows that the top 5 highly correlated features in the dataset. It shows that Packet Length Mean and Average Packet Size have a very strong positive correlation, indicating that these variables very similar information.

7. Data Explorations

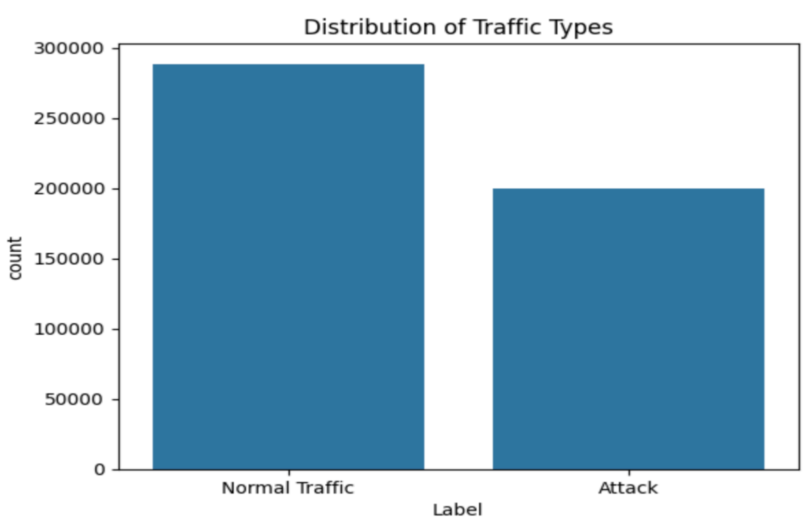
7.1. Bar Chart Analysis

The given bar chart shows the distribution of traffic types in the dataset, categorized as Normal Traffic and Attack. It can observe that normal traffic has a higher frequency (2,88,548) compared to Attack Traffic (1,99,531). This data shows that the dataset contains more normal activity than attack instances.

However, the dataset is slightly imbalanced because the number of normal traffic records is greater than the number of attack records. This type of imbalance is common in real world network traffic datasets. But this bar graph categories both attack network and normal traffic represented in the dataset for further analysis.

```python
#Bar Chart
sns.countplot(x = 'Label',data = new_df)
plt.title("Distribution of Traffic Types")
plt.show()
```

Python




```python
#Count the number of occurences of each category(Normal Traffic and Attack)
new_df['Label'].value_counts()
```

Python

```
Label
Normal Traffic    288548
Attack            199531
Name: count, dtype: int64
```

Figure 32 Bar chart of Traffic Type Distribution

7.2. Pie Chart Analysis

The given pie charts represent the proportion of average Flow Duration and Packet Length Mean between Normal Traffic and Attack categories. From the given charts, it can be observed that both categories contribute almost equally to the average taking around 50% of the total proportion. For Flow Duration, Normal Traffic has a slightly higher share compared to attack indicating longer connection durations in normal traffic. On the other hand, for packet Length Mean, both categories are nearly identical, with Attack being only slightly higher.

Overall, the pie chart shows that there is no significant differences between Normal Traffic and Attack in terms of these features, suggesting similar behavior in Flow duration and Packet size.

```python
#Calculate average values
average_values = new_df.groupby('Label')[['Flow Duration','Packet Length Mean']].mean()
average_values
```

Python

	Flow Duration	Packet Length Mean
Label		
Attack	8.909046e+06	85.573255
Normal Traffic	8.966348e+06	85.542480

```python
#pie chart for flow duration and packet length mean
average_values[['Flow Duration', 'Packet Length Mean']].plot(
    kind='pie',
    subplots=True,
    legend=False,
    autopct='%1.1f%%'
)

plt.show()
```

Python

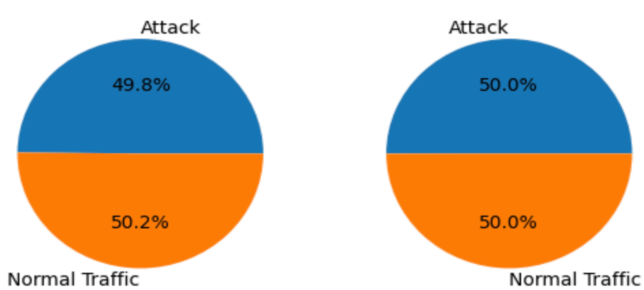


Figure 33 Pie charts of Average Flow Duration and Packet Length Mean

7.3. Box Plot Analysis

The given boxplot compares the distribution of Fwd Packet Length Mean between Normal Traffic and Attack categories. However, it can be observed that both traffic types have very similar distributions, as their median values and spread are very close to each other.

Moreover, the most of data points are concentrated at lower packet length values, while both categories contain many outliers with extremely high packet lengths. Due to the presence of the numerous outliers indicates large variations in packet sizes within the dataset.

Overall, the given boxplot shows that there is no major difference between Normal Traffic and Attack in terms of Fwd packet length mean, since both categories display similar distributions pattern.

```python
#Create box plot
sns.boxplot(x = 'Label',y = 'Fwd Packet Length Mean',data =new_df)
plt.title("Fwd Packet Length Mean by Traffic Type")
plt.show()
```

Python

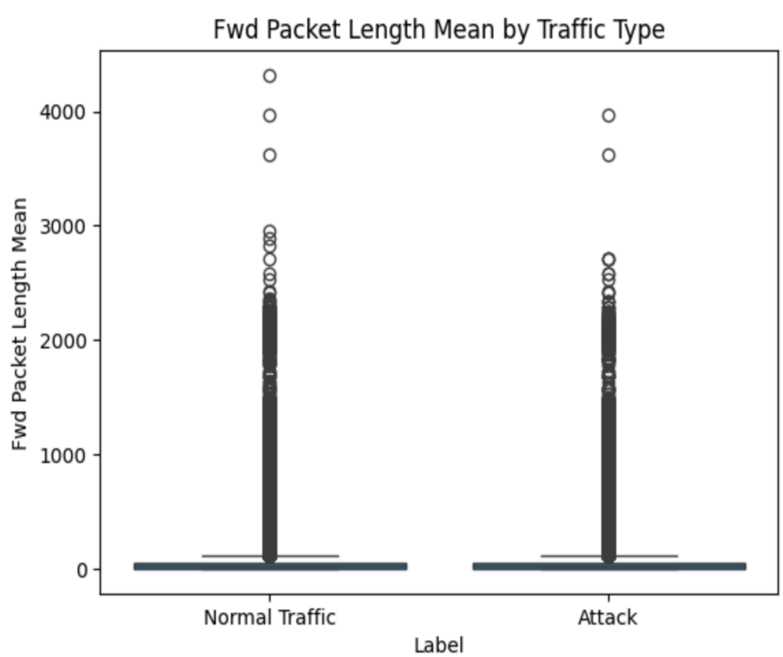


Figure 34 Boxplot of Fwd Length Mean by Traffic Type

7.4. Histogram Analysis

The given histogram shows the distribution of Flow Duration values in the dataset. Most values are concentrated near the lower range, resulting a very high frequency on the left side of the graph.

However, a very few values extend toward higher flow duration ranges that indicates the presence of outliers and positively skewed in the datasets.

```python
# histogram diagram of flow duration
plt.hist(new_df['Flow Duration'], bins=30)
plt.title('Histogram of Flow Duration')
plt.xlabel('Flow Duration')
plt.ylabel('Frequency')
plt.show()
```

[43] 0.0s Python

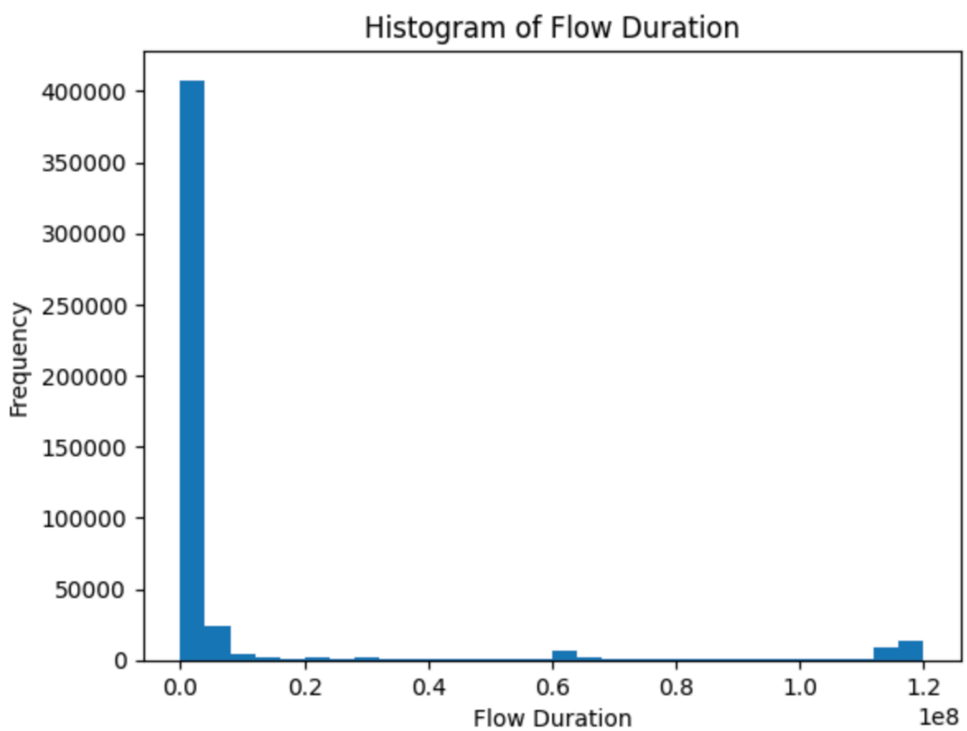


Figure 35 Histogram diagram of flow duration

7.5. Line Graph Analysis

This is the line graph analysis which shows the variation of packet length mean across the first 100 records in the dataset. Most values remain at lower ranges, while a few sharp peaks can be observed at certain indexes.

However, these peaks indicate sudden increases in packet length mean values showing fluctuations and variations in the network traffic behavior. Overall, the graph demonstrates the packet length mean values are not constant and vary across different records in the dataset.

```
#line graph of packet length mean
plt.plot(new_df['Packet Length Mean'].head(100))
plt.title('Line Graph of Packet Length Mean')
plt.xlabel('Index')
plt.ylabel('Packet Length Mean')

plt.show()
```

[44] ✓ 0.0s Python

...

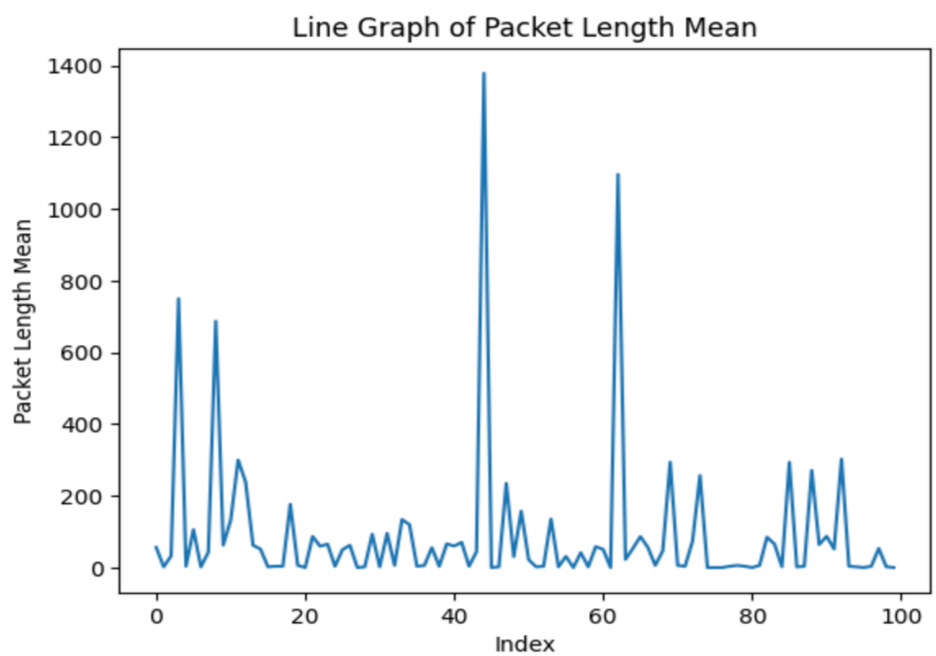


Figure 36 Line diagram of packet length mean

7.6. Scatter Plot Analysis

The given diagram is scatter plot which shows the relationship between Flow Duration and Packet Length Mean. In this diagram most of the data points are concentrated near lower flow duration values, creating a dense cluster on the left side of the graph.

However, the points are widely spread across the plot, indicating no strong linear relationship between Flow duration and packet length mean. A few points with very high values are also visible, showing the presence of outliers in the datasets.

```python
#scatter plot of flow duration and packet length mean
plt.scatter(new_df['Flow Duration'], new_df['Packet Length Mean'])
plt.title('Scatter Plot of Flow Duration vs Packet Length Mean')
plt.xlabel('Flow Duration')
plt.ylabel('Packet Length Mean')

plt.show()
```

[46] ✓ 0.1s Python

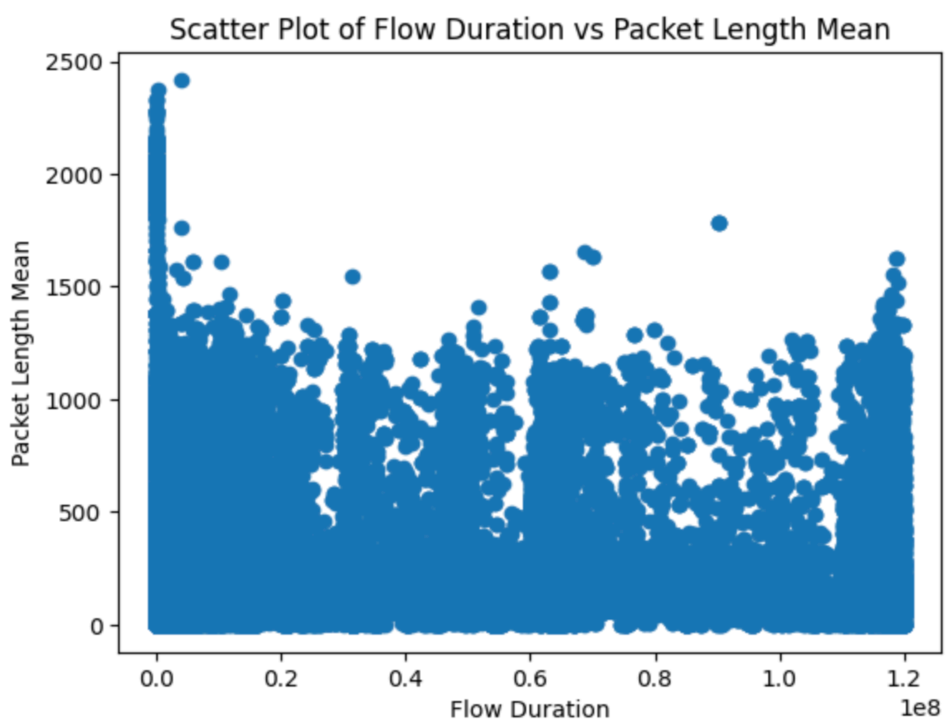


Figure 37 Scatter plot diagram of packet length mean

7.7. HeatMap Analysis

This is the heatmap analysis diagram that shows the correlation between numeric features in the dataset. Lighter colors represent stronger positive correlations, while daker colors represent the weaker relationships. We see that Packet Length Mean, Packet Length std and Average Packet size have strong positive correlations with each other. Meanwhile, other features show weak correlations, indicated by darker shades.

Overall, the heatmap helps to identify relationships between network traffic features in the dataset.

```python
#Create a heatmap of the correlation matrix
correlation_matrix = new_df.corr(numeric_only=True)
plt.figure(figsize=(12,8))
sns.heatmap(correlation_matrix)
plt.title('Correlation Heatmap')
plt.show()
```

[40] 0.1s Python

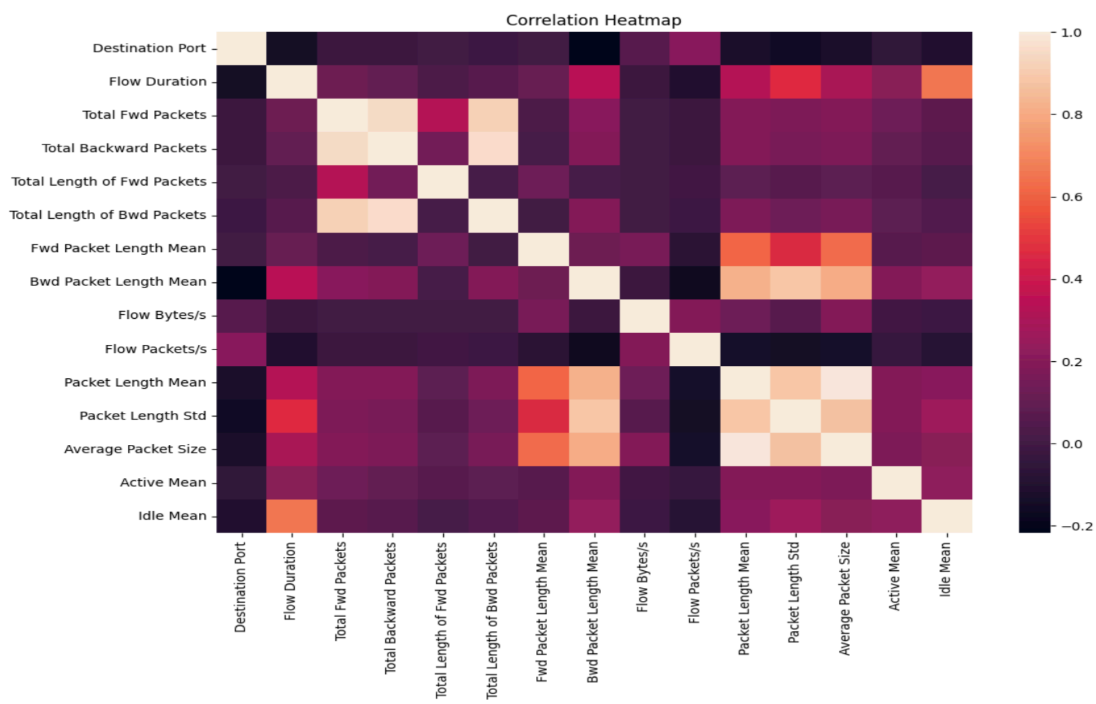


Figure 38 Heatmap diagram of correlations

7.8. Violin Plot Analysis

The given diagram is known as violin plot. It is used to compare the distribution of Packet Length Mean between Normal Traffic and Attack categories. The width of each violin shape represents the density of data values, where wider sections indicate a higher concentration of records.

The both shapes of violin are very similar, indicating that normal traffic and attack have comparable distributions for packet length mean. Overall, the plot shows that there is no major differences between the two traffic categories based on this feature.

```python
#Create a violin plot
sns.violinplot(x='Label', y='Packet Length Mean', data=new_df)
plt.title('Violin Plot of Packet Length Mean by Traffic Type')
plt.show()
```

[39] ✓ 0.3s Python

...

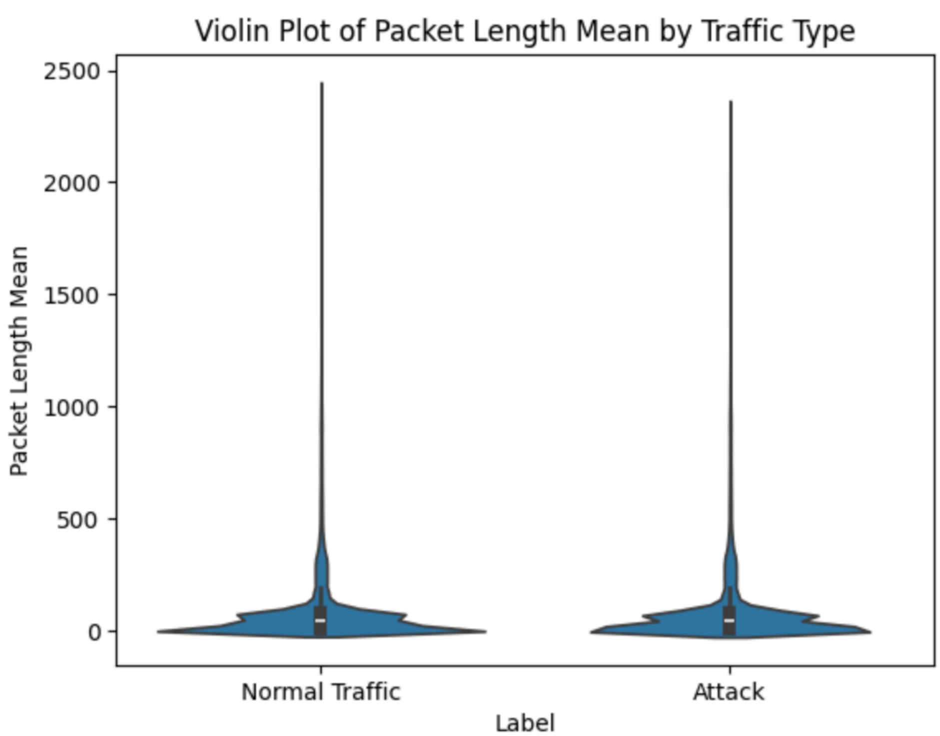


Figure 39 Violin diagram of Packet length mean by traffic type

8. Hypothesis Testing

This is a hypothesis test (independent t-test), a statistical method used to compare the mean values of two groups. In this coursework, it is used to determine whether there is a significant difference in the mean Flow Duration between Normal Traffic and Attack Traffic. The given result shows p-value of 0.474, which is greater than the significance level of 0.05. Therefore, we fail to reject the full hypothesis.

It shows that there is no statistically significant difference in Flow Duration between the two traffic types, meaning both behaves similarity in terms of flow durations.

```python
from scipy.stats import t

# split data
normal = new_df[new_df['Label'] == 'Normal Traffic']['Flow Duration']
attack = new_df[new_df['Label'] == 'Attack']['Flow Duration']

n1, n2 = len(normal), len(attack)
m1, m2 = normal.mean(), attack.mean()
s1, s2 = normal.std(ddof=1), attack.std(ddof=1)

# calculate t-score
t_score = (m1 - m2) / np.sqrt((s1**2 / n1) + (s2**2 / n2))

# critical value
df = n1 + n2 - 2
critical = t.ppf(1 - 0.05/2, df)

# print results
print("t-score:", t_score)
print("critical value:", critical)

if abs(t_score) > critical:
    print("Reject Null Hypothesis")
else:
    print("Accept Null Hypothesis")
```

35] ✓ 0.0s Python

```
t-score: 0.7166329663217962
critical value: 1.9599688449965256
Accept Null Hypothesis
```

Figure 40 Hypothesis test

9. Conclusion

In this coursework, network traffic data was given to analyze network traffic using Python and data analysis libraries such as NumPy, Pandas, Matplotlib, and Seaborn. The main objective of this coursework was to understand the behavior of network traffic and differentiate between normal traffic and attack traffic using statistical and visual analysis techniques.

First, the dataset was prepared by handling missing values, removing duplicates, and selecting important features. This ensured that data was clean and ready to make further analysis. Then, summary statistics such as mean, standard deviation, skewness, and kurtosis were calculated for selected variables (flow duration and packet length mean) to understand the distribution and variability of the data. Correlation analysis was performed to identify relationships between numeric features. It was found that some variables are highly correlated, especially mean packet length and average packet size, indicating redundancy and strong relationships among certain features.

Moreover, in the data exploration phase, different visualization techniques were used. The bar charts illustrate that the dataset is slightly imbalanced with more normal traffic than attack traffic. However, the pie chart shows that both categories have almost equal average flow duration and packet length mean, showing very similar behavior. The boxplot further confirmed that both traffic types have similar distributions with many outliers. At last, a hypothesis test (independent t-test) was conducted to compare the mean flow duration between normal traffic and attack. The output showed no statistically significant difference, confirming that both traffic types behave similarly in terms of flow duration. Overall, this analysis showed that while some differences exist, many features show similar patterns between normal traffic and attack traffic. The cleaned and analyzed dataset is now suitable for future machine learning to train models and improve network intrusion detection and classification.

10. References

GeeksforGeek, 2026. *Numpy Tutorial.* [Online] Available at: https://www.geeksforgeeks.org/python/numpy-tutorial/ [Accessed 23 April 2026].

Pandas, 2026. *Pandas Overview.* [Online] Available at: https://pandas.pydata.org/docs/getting_started/overview.html [Accessed 23 April 2026].

Scalar, 2022. *What is Matplotlib in Python.* [Online] Available at: https://www.scaler.com/topics/matplotlib/matplotlib-in-python/ [Accessed 23 April 2026].

Seaborn, 2025. *seaborn: Statistical data visualizations.* [Online] Available at: https://seaborn.pydata.org/ [Accessed 23 April 2026].

IBM, n.d. *What is Machine Learning.* [Online] Available at: https://www.ibm.com/think/topics/machine-learning [Accessed 23 April 2026].

Python, n.d. *What is Python?.* [Online] Available at: https://www.python.org/doc/essays/blurb/ [Accessed 10 May 2026].

Jupyter, n.d. *Jupyter Notebook.* [Online] Available at: https://jupyter.org/ [Accessed 10 May 2026].

SciPy, n.d. *SciPy.* [Online] Available at: https://docs.scipy.org/doc/scipy/tutorial/index.html#user-guide [Accessed 10 May 2026].

Scikit Learn, n.d. *Scikit-learn: Machine Learning in Python.* [Online] Available at: https://scikit-learn.org/stable/ [Accessed 10 May 2026].